Documentación del proyecto

> *Documento de referencia para la presentación y el mantenimiento del sistema del Colegio María Reina. Conviene completar los datos institucionales entre corchetes y adjuntar las capturas propias indicadas en la sección Evidencias y capturas.*

1. Ficha del proyecto

Dato	Información
Nombre	Sistema de gestión escolar del Colegio María Reina
Institución	Colegio María Reina
Tipo de sistema	Aplicación web con frontend y API
Versión / fecha de entrega	[Completar]
Integrantes y roles	[Completar]
Docente / asignatura	[Completar si corresponde]
Entorno de demostración	[Agregar URL pública, si corresponde]

2. Resumen

El sistema reúne herramientas escolares en una aplicación web con permisos por rol. El personal administrativo gestiona usuarios, materias y clases; los docentes trabajan con sus clases y administran tareas, entregas, calificaciones, asistencia y anuncios; los estudiantes consultan su información académica y realizan entregas.

El proyecto está separado en una interfaz web y una API. La API almacena la información en MariaDB en el desarrollo local y admite PostgreSQL para la demo pública. Los archivos adjuntos se almacenan en el servidor y sus referencias se relacionan con los registros escolares.

3. Objetivos y alcance

Objetivo general

Centralizar procesos académicos y comunicaciones escolares en una plataforma accesible desde el navegador, con vistas y operaciones diferenciadas según el rol de cada usuario.

Alcance funcional implementado

- Inicio público del colegio y acceso de usuarios desde la interfaz.
- Autenticación y rutas privadas para los roles `admin`, `profesor` y `alumno`.
- Gestión administrativa de usuarios, materias, clases y asignaciones de alumnos a clases.
- Herramientas docentes para gestionar tareas y entregas, calificaciones, asistencia y anuncios.
- Herramientas del alumno para consultar materias/clases, tareas, asistencia, boletín y anuncios, y entregar trabajos.
- Flujo de solicitud de recuperación y restablecimiento de contraseña.
- Persistencia relacional para usuarios, materias, clases y módulos escolares.

- Carga de archivos en el backend para adjuntos de tareas y entregas.

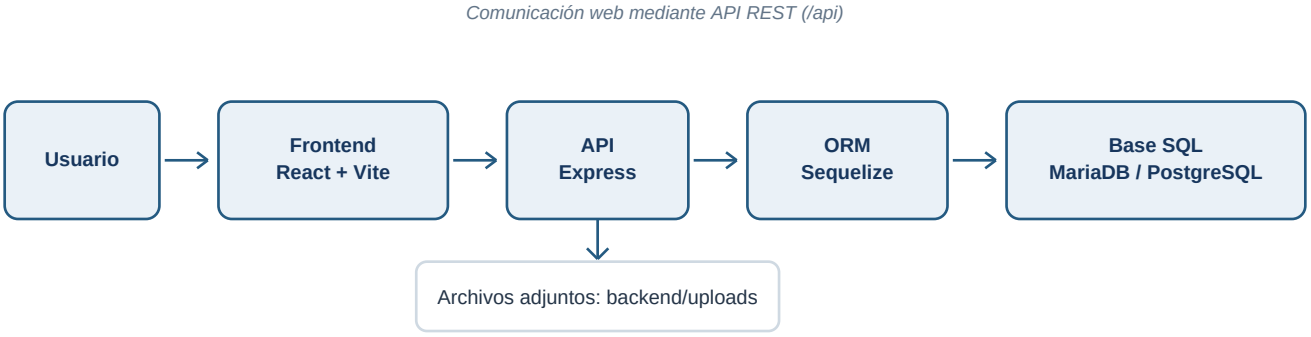
> *Para describir funcionalidades no enumeradas aquí, verificar primero que estén disponibles en la versión que se presenta. La existencia de una pantalla o ruta no garantiza por sí sola que un servicio externo (por ejemplo, correo) esté configurado.*

4. Usuarios y permisos

Rol	Funciones principales
Administrador	Gestión de usuarios, materias, clases, asignaciones, asistencias y revisión/auditoría de notas.
Profesor	Consulta de sus clases y alumnos; publicación de tareas y anuncios; revisión de entregas; carga de notas y asistencia.
Alumno	Consulta de sus clases y materias, tareas, asistencia, boletín y anuncios; entrega de trabajos.

Las páginas privadas se protegen en el frontend según el rol y las operaciones se realizan contra rutas de la API. Al preparar la presentación, usar cuentas de demostración y datos ficticios, no credenciales ni información personal de alumnos reales.

5. Arquitectura



- **Frontend:** aplicación de página única; usa React Router para las vistas y Axios para comunicarse con la API. `VITE_API_URL` configura la URL del backend; en desarrollo, el valor predeterminado es `http://localhost:5000`.
- **Backend:** Express monta las rutas bajo `/api`, valida la conexión a la base de datos al iniciar y publica los adjuntos bajo `/uploads`.
- **Autenticación:** el frontend adjunta el token guardado en el navegador a las solicitudes autenticadas. El backend usa JWT y contraseñas con hash.
- **Base de datos:** Sequelize permite MariaDB local y PostgreSQL para la demo. Las migraciones crean el esquema; la API no debe considerarse un sustituto de las migraciones.
- **Demo alojada:** Render ejecuta la API y publica el frontend estático. La persistencia de archivos en el plan gratuito puede ser efímera; consultar DEMO-DEPLOYMENT.md.

6. Módulos y rutas principales de la API

Las rutas se montan bajo el prefijo `/api`. La tabla resume áreas funcionales; consultar las rutas y controladores del backend para los métodos, parámetros y reglas exactas.

Prefijo	Área
`/api/auth`	Inicio de sesión y recuperación/restablecimiento de contraseña.
`/api/usuarios`	Consulta y administración de usuarios.
`/api/materias`	Materias y consultas relacionadas con docentes/alumnos.
`/api/clases`	Clases, horarios y asignaciones.
`/api/tareas` y `/api/tareas/alumno`	Tareas y consultas del alumno.
`/api/entregas`	Entregas de tareas.
`/api/calificaciones` y `/api/notas`	Calificaciones, notas trimestrales y auditoría relacionada.
`/api/asistencias`	Asistencia por clase.
`/api/anuncios`	Anuncios escolares.
`/api/cuotas`	Cuotas.
`/api/admin`	Estadísticas y funciones administrativas.

La ruta raíz `/` sirve como comprobación básica del backend e indica qué dialecto de base de datos está activo.

7. Base de datos

Motor y preparación

- **Desarrollo local:** MariaDB por defecto. La conexión se configura en `proyectocolegio-backend/.env`; nunca subir ese archivo al repositorio.
- **Demo pública:** PostgreSQL mediante `DB_DIALECT=postgres` y `DATABASE_URL`.
- **Migraciones locales:** `npm run db:create:local` prepara la base si hace falta y `npm run migrate:academics` crea las tablas del sistema académico/escolar.
- **Datos iniciales:** `npm run seed:local` y `npm run seed:academics` agregan datos de prueba local; no sustituyen un respaldo ni deben ejecutarse con datos productivos sin revisar su propósito.
- El directorio `proyectocolegio-backend/data/db` contiene archivos de almacenamiento de MongoDB heredados; no es el esquema SQL que usan las instrucciones actuales de MariaDB/PostgreSQL, y la preparación local no importa registros desde allí.

Entidades y relaciones principales

El siguiente esquema es conceptual y resume las claves foráneas que aparecen en las migraciones. Los nombres `school_*` corresponden a las tablas relacionales de los módulos escolares.

Entidad origen	Relación	Entidad destino
users	profesor asignado	materias
materias	organiza	clases
clases	asigna	clase_profesores
users	docente	clase_profesores
clases	inscribe	clase_alumnos
users	alumno	clase_alumnos
materias	tiene	school_tareas

Entidad origen	Relación	Entidad destino
users	profesor crea	school_tareas
school_tareas	recibe	school_entregas
users	entrega	school_entregas
clases	registra	school_asistencias
users	alumno	school_asistencias
materias	evalua	school_calificaciones
users	recibe	school_calificaciones
materias	agrupa	school_notas_trimestrales
users	recibe	school_notas_trimestrales
school_notas_trimestrales	historial	school_auditoria_notas
materias	publica	school_anuncios
users	autor	school_anuncios
users	adeuda	school_cuotas
users	registra	school_asistencia_general

Tabla / grupo	Propósito
`users`	Cuentas, rol, datos de curso y atributos de autenticación.
`materias`, `clases`	Oferta por curso, horario y aula.
`clase_profesores`, `clase_alumnos`	Relaciones de asignación entre clases, profesores y alumnos.
`school_asistencias`, `school_asistencia_general`	Asistencia asociada a una clase y asistencia general del alumno, respectivamente.
`school_calificaciones`, `school_notas_trimestrales`, `school_auditoria_notas`	Evaluaciones, notas trimestrales y registro de cambios de notas.
`school_tareas`, `school_entregas`	Actividades publicadas y archivos entregados por alumnos.
`school_anuncios`	Anuncios asociados a materia y profesor.
`school_cuotas`	Cuotas asociadas a usuarios.

La estructura exacta, los campos, las restricciones y las acciones ante borrado se definen en `proyectocolegio-backend/src/database/migrations/`. Para una memoria formal, generar el diagrama directamente desde la base preparada (por ejemplo, con DBeaver o MySQL Workbench) y adjuntar una captura legible; el esquema conceptual de arriba no reemplaza el diagrama físico.

Respaldo y privacidad

Antes de modificar o limpiar una base, generar un respaldo y comprobar cómo restaurarlo. No incluir en la memoria capturas de contraseñas, tokens, datos de contacto, DNIs, archivos de alumnos ni registros reales identificables. Recortar o anonimizar esos datos y usar el conjunto ficticio de la demo cuando sea posible.

8. Instalación y ejecución local

Requisitos

- Node.js y npm compatibles con los paquetes del proyecto.

- MariaDB disponible localmente o en un entorno de desarrollo autorizado.
- Git para clonar el repositorio.

Pasos

1. Crear `proyectocolegio-backend/.env` con la conexión MariaDB (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`) y un `JWT_SECRET` no trivial. No compartir ni versionar secretos.

2. Preparar el esquema y los datos de desarrollo:

```
cd proyectocolegio-backend
npm install
npm run db:create:local
npm run migrate:academics
npm run seed:local
npm run seed:academics
```

3. En otra terminal, ejecutar la API:

```
cd proyectocolegio-backend
npm run dev
```

4. Ejecutar el frontend en otra terminal:

```
cd proyectoColegio-frontend
npm install
npm run dev
```

5. Abrir la URL local que Vite informa en la terminal. Si el backend no está en `http://localhost:5000`, establecer `VITE_API_URL` en el entorno del frontend antes de iniciarlo.

En Windows, los comandos `cd` anteriores se ejecutan en PowerShell. Para los comandos de npm también se puede abrir una terminal directamente en la carpeta indicada.

9. Despliegue de demostración

El despliegue de demostración está definido como Blueprint en `render.yaml`. Requiere un PostgreSQL separado y variables configuradas en Render; no reutilizar secretos, usuarios ni una base real del colegio. Seguir los pasos y las limitaciones detalladas en DEMO-DEPLOYMENT.md.

En particular, las cuentas de demo se configuran mediante variables de entorno y el seed está limitado a PostgreSQL con datos ficticios. Si se necesita recuperación por correo, también debe configurarse un servicio SMTP; no asumir que funciona solo porque existe la pantalla.

10. Pruebas y controles

Ejecutar desde la carpeta correspondiente:

```
# Backend
npm test
npm run test:postgres-compat

# Frontend
npm run lint
npm run build
```

Registrar para la entrega la fecha, el entorno, el resultado y cualquier error pendiente. No presentar una prueba como exitosa si no se ejecutó en la versión que se entrega.

11. Evidencias y capturas

Guardar capturas propias dentro de `docs/capturas/` y reemplazar `[Pendiente]` con enlaces Markdown relativos a cada imagen. No se agregan capturas de ejemplo en este documento para evitar presentar imágenes inventadas o datos de personas reales.

ID	Captura recomendada	Qué debe demostrar	Nombre sugerido
E01	Inicio público	Portada/navegación principal en navegador.	`01-inicio.png`
E02	Acceso	Formulario de inicio de sesión; ocultar credenciales antes de capturar.	`02-acceso.png`
E03	Panel administrador	Resumen y navegación administrativa.	`03-panel-admin.png`
E04	Gestión de usuarios	Listado con datos ficticios y roles visibles.	`04-usuarios.png`
E05	Materias y clases	Materias, horarios o asignaciones ficticias.	`05-materias-clases.png`
E06	Panel docente	Clases y herramientas del profesor.	`06-panel-docente.png`
E07	Tareas y entregas	Publicación, entrega o revisión de una actividad de prueba.	`07-tareas-entregas.png`
E08	Notas y auditoría	Una nota ficticia y, si corresponde, su historial de cambios.	`08-notas-auditoria.png`
E09	Asistencia	Registro o consulta con alumnos ficticios.	`09-asistencia.png`
E10	Panel alumno	Materias, tareas, boletín, anuncios o asistencia del alumno demo.	`10-panel-alumno.png`
E11	Esquema de base de datos	Diagrama físico exportado desde la base local/demo, con tablas y relaciones legibles.	`11-esquema-bd.png`
E12	Tablas de base de datos	Vista de las tablas relevantes; ocultar filas/datos personales.	`12-tablas-bd.png`
E13	Pruebas	Terminal con comandos y resultados ejecutados, sin rutas o variables sensibles visibles.	`13-pruebas.png`
E14	Demo desplegada	Sitio accesible y URL visible, si existe una demo activa.	`14-demo.png`

Cómo capturar la base de datos

1. Conectar DBeaver o MySQL Workbench a la base de desarrollo autorizada, nunca a producción para la presentación.
2. Generar un diagrama ER desde el esquema y encuadrar tablas/relaciones para que nombres y cardinalidades se lean.
3. Capturar otra vista de las tablas principales. Mostrar nombres de columnas; si hacen falta filas de ejemplo, usar únicamente datos ficticios.
4. Revisar cada imagen antes de guardarla: eliminar o tapar contraseñas, tokens, datos personales, URLs privadas y credenciales de conexión.
5. Guardar las imágenes como PNG en `docs/capturas/` y enlazarlas en esta sección o en el informe final.

12. Limitaciones conocidas y trabajo futuro

- El almacenamiento local de adjuntos no debe considerarse persistente en los discos efímeros del plan gratuito de Render.

- La recuperación de contraseña por correo depende de la configuración SMTP del entorno.
- La base demo es compartida: las personas que acceden pueden modificar sus datos ficticios.
- Los archivos de MongoDB presentes en `data/db` no son el origen de las bases SQL configuradas para desarrollo y demo.
- [Agregar aquí limitaciones funcionales verificadas y mejoras priorizadas.]

13. Lista final para la entrega

- ■ Completar integrantes, roles, versión, fecha y datos de presentación.
- ■ Revisar cada afirmación funcional en la versión que se va a entregar.
- ■ Adjuntar las capturas E01–E14 que correspondan; retirar de la lista las que no apliquen.
- ■ Agregar un diagrama físico actualizado de la base de datos.
- ■ Confirmar que las capturas solo usan información ficticia o anonimizada.
- ■ Ejecutar y anotar los resultados de las pruebas.
- ■ Revisar que no se incluyan `.env`, credenciales, tokens ni datos reales.
- ■ Confirmar las URLs y limitaciones de la demo, si se presenta.